

第四章 策略梯度

策略梯度算法

想干什么？ 找一个好的策略，输入状态，输出采取各动作的概率，最大化奖励。

但是，这里不学习 Q ，也不计算某个动作的奖励最多，而是直接调整策略网络，让策略自己变好。

一些概念

用神经网络表示策略： π_θ ，其中 θ 是神经网络里面的参数，也是我们真正要训练的东西。策略梯度并不是直接修改动作，而是修改神经网络参数，使网络以后输出不同的动作概率。

轨迹 τ ：从开始到结束每一步的状态、动作的顺序集合。

回报 $R(\tau)$ ：奖励之和。

$J(\theta)$ ：以 θ 为神经网络参数的策略平均获得的奖励值（奖励的期望）。

记为

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[R(\tau)]$$

理解这个公式： $J(\theta)$ 是 $R(\tau)$ 的期望，这个期望是在 τ 服从 $p_\theta(\tau)$ 概率分布的情况下的。

而 $p_\theta(\tau)$ 的意思是在 θ 参数下出现 τ 轨迹的概率分布。

（我们的目标就是找到 θ 使得 $J(\theta)$ 最大化。）

核心直觉：我们希望通过调整 θ ，让高回报的轨迹更多出现，而低回报的轨迹少出现。

现在关注一条轨迹是怎么形成的：实际上，我们有

$$p_\theta(\tau) = p(s_1) \prod_t \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

取对数，有

$$\log p_\theta(\tau) = \log p(s_1) + \sum_t \log \pi_\theta(a_t | s_t) + \sum_t \log p(s_{t+1} | s_t, a_t)$$

我们发现等号右边的第一项和第三项都和策略的参数 θ 无关，所以当我们优化策略时，对两边对 θ 求梯度，得到：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

也就是说，轨迹中环境产生的部分不用反向传播，最终只需要计算策略网络对“自己采取的动作”给出的对数概率。

再看回报：我们有

$$J(\theta) = \sum_{\tau} R(\tau) p_{\theta}(\tau)$$

对 θ 求梯度：

$$\nabla_{\theta} J(\theta) = \sum_{\tau} R(\tau) \nabla_{\theta} p_{\theta}(\tau)$$

使用一个数学恒等式：

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

就可以得到：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)]$$

再把轨迹概率拆开：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

也即

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

理解一个点： $\nabla_{\theta} f$ 是一个矢量，指示一个方向：当前如果沿着这个方向改动 θ ，能最快让 θ 的值提升。对于 $\nabla_{\theta} J(\theta)$ ，就是一个能让 $J(\theta)$ 最快提升的 θ 的改变方向。

这正是我们要求的。然后看右边的部分，可以分成三层来考虑：

$$\mathbb{E}_{\tau \sim p_{\theta}} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

最外层的期望就是多次采样轨迹平均，很好理解；

期望里面：对于每一次的轨迹，有 $R(\tau)$ 反映这次是好是坏，而后面的求和项根据上面的推导可以知道，是“让这次轨迹出现概率最快提高的 θ 的改变方向”。

可以把 $R(\tau)$ 理解成后面的权重： R 越大，说明这条轨迹越好，应当大力把 θ 往这个方向拉；反之亦然。

总结一下，让 $J(\theta)$ 提升最快的方向，是“不同轨迹出现概率提升最快的方向”以“该轨迹的回报”为权重加权平均得到的。

思考 1：这个方法有一个很显而易见的问题：一条轨迹的回报是由很多步动作的奖励相加得来的，整条轨迹的回报高不代表每一步的回报都高，奖励高回报轨迹同时也会奖励其中可能存在的坏动作。

思考 2：判断是奖励还是惩罚看的是 $R(\tau)$ 的值，只要看到 $R(\tau) > 0$ 就奖励这个 τ ，即使目前的 $J(\theta) > R(\tau)$ ，会把 J 往更低的方向去拉。

以上的两个思考是很常见的想法。实际上，当前的公式没有非常严重的问题，能够完成优化的目标。核心思想在于多次试验的平均以及不同方向的抵消：对于思考 1，我们优化的目标就是提升 J ，完全可以无视 J 内部的构成。只要还在优化，坏动作就不可能一直被奖励。事实上，后面会提到一种方法，把优化细化到每一个动作。这样做只是提升了优化的效率（减小了方差）。

对于思考 2，事实上，对于单次动作，确实是只要 $R > 0$ 就会奖励对应 τ 的动作，但在多次试验的情况下，低 R 动作的奖励会被高 R 动作的奖励拉过去，最终的效果是一样的。

举个例子： θ 只有一个参数（标量），两个轨迹的 R 分别是 10 和 20，当前的 J 为 15。

单算两条轨迹的梯度，可能得到 -1.2 和 $+2$ 。

如果进行一个优化：把 R 换成 $R - J$ ，得到 $+0.3$ 和 $+0.5$ 。

发现：两者总和是一样的，只不过方差会大幅减小，更加稳定。

注：这样优化可行、符合数学原理解释：

令 $b \approx \mathbb{E}[R(\tau)]$ ，则有

$$\nabla J = \mathbb{E}[R(\tau)g(\tau)]$$

现在改成

$$\mathbb{E}[(R(\tau) - b)g(\tau)] = \mathbb{E}[R(\tau)g(\tau)] - b\mathbb{E}[g(\tau)]$$

会发现第二项为 0，原式仍然成立。

实际上我们不可能遍历所有的 τ ，求右侧的期望转化为多次采样求平均：跑很多次，得到多条轨迹 $\tau^1, \tau^2, \dots, \tau^N$ ，用它们的平均值近似真正的期望：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{n=1}^N \sum_t R(\tau^{(n)}) \nabla_{\theta} \log \pi_{\theta}(a_t^{(n)} | s_t^{(n)})$$

实际的训练流程：

先有一个基础策略 θ_0 ，在该策略下跑若干局，记录

$$R(\tau) \sum_t \log \pi_{\theta}(a_t | s_t)$$

然后将其交给深度学习优化器，计算其对 θ 的反向传播，即

$$R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

多条线路都计算后求平均即得到了

$$\hat{g} \approx \nabla_{\theta} J(\theta_0)$$

然后更新策略：

$$\theta_1 = \theta_0 + \alpha \hat{g}$$

完成一次从采样、反向传播到更新的循环。

注意：在更新完策略进入下一轮时，需要重新采样而不能直接沿用之前的旧数据（因为之前的旧数据是在原策略下采集的，梯度估计要求轨迹来自当前正在优化的策略）。

优化

回顾一下上面的两个思考，从中我们可以得出两种优化的方法：

1. **添加基线 baseline**：给表现一个参照，来区分好和坏。

也就是上面提到的，把

$$R(\tau)$$

换成

$$R(\tau) - b$$

其中 b 就是我们所说的 baseline。

如何确定这个 b 呢？直观感觉是平均水平：如果一条轨迹的表现比平均水平好，就应该向其学习，也就是最简单的基线是所有轨迹回报的平均值：

$$b \approx \mathbb{E}[R(\tau)]$$

2. 细化：从“一整条轨迹”到“每一步动作”。

之前用

$$R(\tau) \sum_t \log \pi_{\theta}(a_t | s_t)$$

权重 $R(\tau)$ 是一整条轨迹的回报。

现在细化为

$$\sum_t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

其中 G_t 为第 t 时刻动作的未来回报：

$$G_t = r_{t+1} + r_{t+2} + \cdots + r_T$$

t 时刻前的回报不应该成为影响 t 时刻决策的因素。

注意这里换成 G_t 以后不能再提到求和号前面，因为已经和 t 有关。

也可以使用带有折扣因子的版本，更注重短期收益：

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$$

注意这里如果换成带有折扣因子的版本，实际上训练目标已经改变。但这是正常的取舍：更重视近期奖励，让训练过程更加稳定。

同时我们发现，细化到每一步动作后，之前我们采取的 baseline

$$b \approx \mathbb{E}[R(\tau)]$$

有些粗糙了。

所以我们将 baseline 也细化成每一步动作的版本：

$$b(s_t)$$

一般而言，这个状态相关的基线通常接近价值函数，即

$$b(s_t) \approx V^{\pi}(s_t)$$

也就是当前策略从状态 s_t 出发，平均能够得到的未来回报。

把上面的结合在一起看：我们把一开始的 $R(\tau)$ 换成

$$G_t - b(s_t)$$

定义优势函数 advantage 为：

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

最终的策略梯度形式就是：

$$\nabla_\theta J(\theta) \approx \sum_t (G_t - b(s_t)) \nabla_\theta \log \pi_\theta(a_t | s_t)$$

或者简写为：

$$\nabla_\theta J(\theta) \approx \sum_t A(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t)$$

其中理解：

- $Q^\pi(s, a)$ ：在状态 s 执行动作 a 后，未来平均能得到多少回报；
- $V^\pi(s)$ ：在状态 s 中按照当前策略平均能得到多少回报；
- $A^\pi(s, a)$ ：动作 a 相对于当前状态平均水平有多好。